Scripts do Banco

Migrations

1. V1__initial_schema.sql

```
-- ==========================================================
-- Sistema de Pré-Matrícula Acadêmica
-- Migration: V1__initial_schema.sql
-- Descrição: Criação do esquema inicial do banco de dados.
-- ==========================================================

-- Extensão para geração de UUID
CREATE EXTENSION IF NOT EXISTS "pgcrypto";


-- ======================================================
-- USERS
-- ======================================================

CREATE TABLE users (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    full_name VARCHAR(150) NOT NULL,

    email VARCHAR(150) NOT NULL UNIQUE,

    password VARCHAR(255) NOT NULL,

    role VARCHAR(20) NOT NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    active BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT chk_user_role
        CHECK (role IN ('ADMIN', 'STUDENT'))

);


CREATE TABLE admins (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id UUID NOT NULL UNIQUE,
```

```
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    active BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT fk_admin_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)

);


CREATE TABLE students (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    user_id UUID NOT NULL UNIQUE,

    registration_number VARCHAR(20) NOT NULL UNIQUE,

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    active BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT fk_student_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)

);


-- ======================================================
-- ACADEMIC DOMAIN
-- ======================================================

CREATE TABLE academic_periods (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    code VARCHAR(20) NOT NULL UNIQUE,

    name VARCHAR(100) NOT NULL,

    start_date DATE NOT NULL,

    end_date DATE NOT NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```sql
    active BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT chk_period_dates
        CHECK (start_date <= end_date)

);


CREATE TABLE disciplines (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    code VARCHAR(20) NOT NULL UNIQUE,

    name VARCHAR(120) NOT NULL,

    workload INTEGER NOT NULL,

    prerequisites TEXT,

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    active BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT chk_discipline_workload
        CHECK (workload > 0)

);


CREATE TABLE classes (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    code VARCHAR(20) NOT NULL,

    name VARCHAR(150) NOT NULL,

    discipline_id UUID NOT NULL,

    academic_period_id UUID NOT NULL,

    vacancies INTEGER NOT NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    active BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT fk_class_discipline
```

```
        FOREIGN KEY (discipline_id)
        REFERENCES disciplines(id),

    CONSTRAINT fk_class_academic_period
        FOREIGN KEY (academic_period_id)
        REFERENCES academic_periods(id),

    CONSTRAINT uk_class_code_period
        UNIQUE (code, academic_period_id),

    CONSTRAINT chk_class_vacancies
        CHECK (vacancies > 0)

);


-- ======================================================
-- ENROLLMENT PROCESS
-- ======================================================

CREATE TABLE enrollment_processes (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    title VARCHAR(150) NOT NULL,

    academic_period_id UUID NOT NULL,

    start_date DATE NOT NULL,

    end_date DATE NOT NULL,

    allow_over_capacity BOOLEAN NOT NULL DEFAULT FALSE,

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    active BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT fk_process_period
        FOREIGN KEY (academic_period_id)
        REFERENCES academic_periods(id),

    CONSTRAINT chk_process_dates
        CHECK (start_date <= end_date)

);


CREATE TABLE enrollment_process_classes (

    enrollment_process_id UUID NOT NULL,
```

```
    class_id UUID NOT NULL,

    PRIMARY KEY (
        enrollment_process_id,
        class_id
    ),

    CONSTRAINT fk_epc_process
        FOREIGN KEY (enrollment_process_id)
        REFERENCES enrollment_processes(id),

    CONSTRAINT fk_epc_class
        FOREIGN KEY (class_id)
        REFERENCES classes(id),

    -- Cada turma pode pertencer a apenas um processo de pré-matrícula.
    CONSTRAINT uk_class_single_process
        UNIQUE(class_id)

);


-- ======================================================
-- ENROLLMENTS
-- ======================================================

CREATE TABLE enrollments (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    student_id UUID NOT NULL,

    enrollment_process_id UUID NOT NULL,

    class_id UUID NOT NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    active BOOLEAN NOT NULL DEFAULT TRUE,

    CONSTRAINT fk_enrollment_student
        FOREIGN KEY (student_id)
        REFERENCES students(id),

    CONSTRAINT fk_enrollment_process
        FOREIGN KEY (enrollment_process_id)
        REFERENCES enrollment_processes(id),

    CONSTRAINT fk_enrollment_class
        FOREIGN KEY (class_id)
        REFERENCES classes(id),
```

```
    -- o aluno não pode se inscrever duas vezes na mesma turma dentro do mesmo processo de matrícula
    CONSTRAINT uk_student_process_class
        UNIQUE (
            student_id,
            enrollment_process_id,
            class_id
        )

);
```

2. V2__add_allow_oversubscription_to_classes.sql

```
ALTER TABLE classes ADD COLUMN allow_oversubscription BOOLEAN NOT NULL DEFAULT FALSE;
```

3. V3__update_users_for_first_access.sql

```
ALTER TABLE users
    ALTER COLUMN password DROP NOT NULL;

ALTER TABLE users
    ADD COLUMN first_access BOOLEAN NOT NULL DEFAULT TRUE;
```

4. V4__create_verification_token_table.sql

```
CREATE TABLE verification_tokens (
    id UUID PRIMARY KEY,
    token VARCHAR(255) NOT NULL UNIQUE,
    token_type VARCHAR(50) NOT NULL,
    expires_at TIMESTAMP NOT NULL,
    used BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP NOT NULL,
    user_id UUID NOT NULL,

    CONSTRAINT fk_verification_token_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)
```

```
);
```

5. V5__insert_initial_admin.sql

```sql
-- Initial administrator
-- Email: admin@prematricula.uesb.br
-- Password: Admin@123

WITH new_admin AS (

    INSERT INTO users (
        id,
        full_name,
        email,
        password,
        first_access,
        role,
        created_at,
        updated_at,
        active
    )
    VALUES (
        gen_random_uuid(),
        'Administrador do Sistema',
        'admin@prematricula.uesb.br',
        '$2a$10$mWXPawON0QKxOIC05lkh9OlBH5ThS0/eAoNlwHtSndQwYu9POfgYe',
        FALSE,
        'ADMIN',
        CURRENT_TIMESTAMP,
        CURRENT_TIMESTAMP,
        TRUE
    )
    RETURNING id

)

INSERT INTO admins (
    id,
    user_id,
    created_at,
    updated_at,
    active
)
SELECT
    gen_random_uuid(),
    id,
    CURRENT_TIMESTAMP,
    CURRENT_TIMESTAMP,
    TRUE
```

```sql
FROM new_admin;
```

6. V6__refactor_enrollment_model.sql

```sql
-- ========================================================
-- 1. AJUSTAR ENROLLMENT_PROCESS_CLASSES
-- ========================================================

-- A entity EnrollmentProcessClass utiliza um UUID próprio.
ALTER TABLE enrollment_process_classes
ADD COLUMN id UUID NOT NULL DEFAULT gen_random_uuid();

-- A tabela antiga utiliza chave primária composta.
ALTER TABLE enrollment_process_classes
DROP CONSTRAINT enrollment_process_classes_pkey;

-- class_id passa a acompanhar o nome usado pela entity.
ALTER TABLE enrollment_process_classes
RENAME COLUMN class_id TO class_group_id;

-- Campos de auditoria e exclusão lógica utilizados pela entity.
ALTER TABLE enrollment_process_classes
ADD COLUMN created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP;

ALTER TABLE enrollment_process_classes
ADD COLUMN updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP;

ALTER TABLE enrollment_process_classes
ADD COLUMN active BOOLEAN NOT NULL DEFAULT TRUE;

-- Nova chave primária.
ALTER TABLE enrollment_process_classes
ADD CONSTRAINT pk_enrollment_process_classes
    PRIMARY KEY (id);

-- Um mesmo processo não pode conter a mesma turma duas vezes.
ALTER TABLE enrollment_process_classes
ADD CONSTRAINT uk_enrollment_process_class
    UNIQUE (
        enrollment_process_id,
        class_group_id
    );


-- ========================================================
-- 2. REFATORAR ENROLLMENTS
-- ========================================================

-- A antiga tabela enrollments representava uma turma escolhida.
```

```
-- Agora ela representará a pré-matrícula completa do aluno.

ALTER TABLE enrollments
DROP CONSTRAINT fk_enrollment_class;

ALTER TABLE enrollments
DROP CONSTRAINT uk_student_process_class;

ALTER TABLE enrollments
DROP COLUMN class_id;

-- Um aluno possui somente uma pré-matrícula por processo.
ALTER TABLE enrollments
ADD CONSTRAINT uk_student_enrollment_process
    UNIQUE (
        student_id,
        enrollment_process_id
    );


-- ========================================================
-- 3. CRIAR ENROLLMENT_ITEMS
-- ========================================================

CREATE TABLE enrollment_items (

    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

    enrollment_id UUID NOT NULL,

    enrollment_process_class_id UUID NOT NULL,

    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_enrollment_item_enrollment
        FOREIGN KEY (enrollment_id)
        REFERENCES enrollments(id),

    CONSTRAINT fk_enrollment_item_process_class
        FOREIGN KEY (enrollment_process_class_id)
        REFERENCES enrollment_process_classes(id),

    CONSTRAINT uk_enrollment_item_process_class
        UNIQUE (
            enrollment_id,
            enrollment_process_class_id
        )
);


-- ========================================================
-- 4. ÍNDICES
```

```sql
-- ======================================================

CREATE INDEX idx_enrollments_process
    ON enrollments(enrollment_process_id);

CREATE INDEX idx_enrollment_items_process_class
    ON enrollment_items(enrollment_process_class_id);
```